



NPTEL Course on “Computer System Optimization for ML”

Lecture #11



Agenda

- **Recap**
- **Dataflow for DNN Execution**

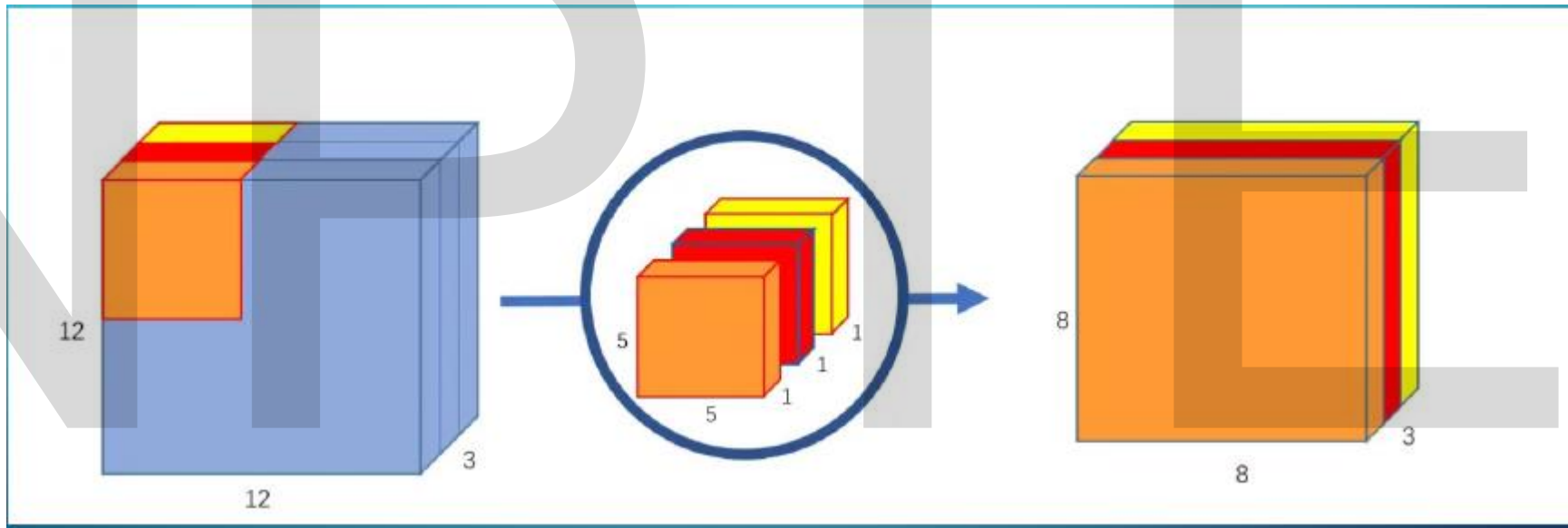
NPTREL

Summary of Popular DNNs

Metrics	LeNet-5	AlexNet	VGG-16	GoogLeNet (v1)	ResNet-50
Top-5 error	n/a	16.4	7.4	6.7	5.3
Input Size	28x28	227x227	224x224	224x224	224x224
# of CONV Layers	2	5	16	21 (depth)	49
Filter Sizes	5	3, 5, 11	3	1, 3, 5, 7	1, 3, 7
# of Channels	1, 6	3 - 256	3 - 512	3 - 1024	3 - 2048
# of Filters	6, 16	96 - 384	64 - 512	64 - 384	64 - 2048
Stride	1	1, 4	1	1, 2	1, 2
# of Weights	2.6k	2.3M	14.7M	6.0M	23.5M
# of MACs	283k	666M	15.3G	1.43G	3.86G
# of FC layers	2	3	3	1	1
# of Weights	58k	58.6M	124M	1M	2M
# of MACs	58k	58.6M	124M	1M	2M
Total Weights	60k	61M	138M	7M	25.5M
Total MACs	341k	724M	15.5G	1.43G	3.9G

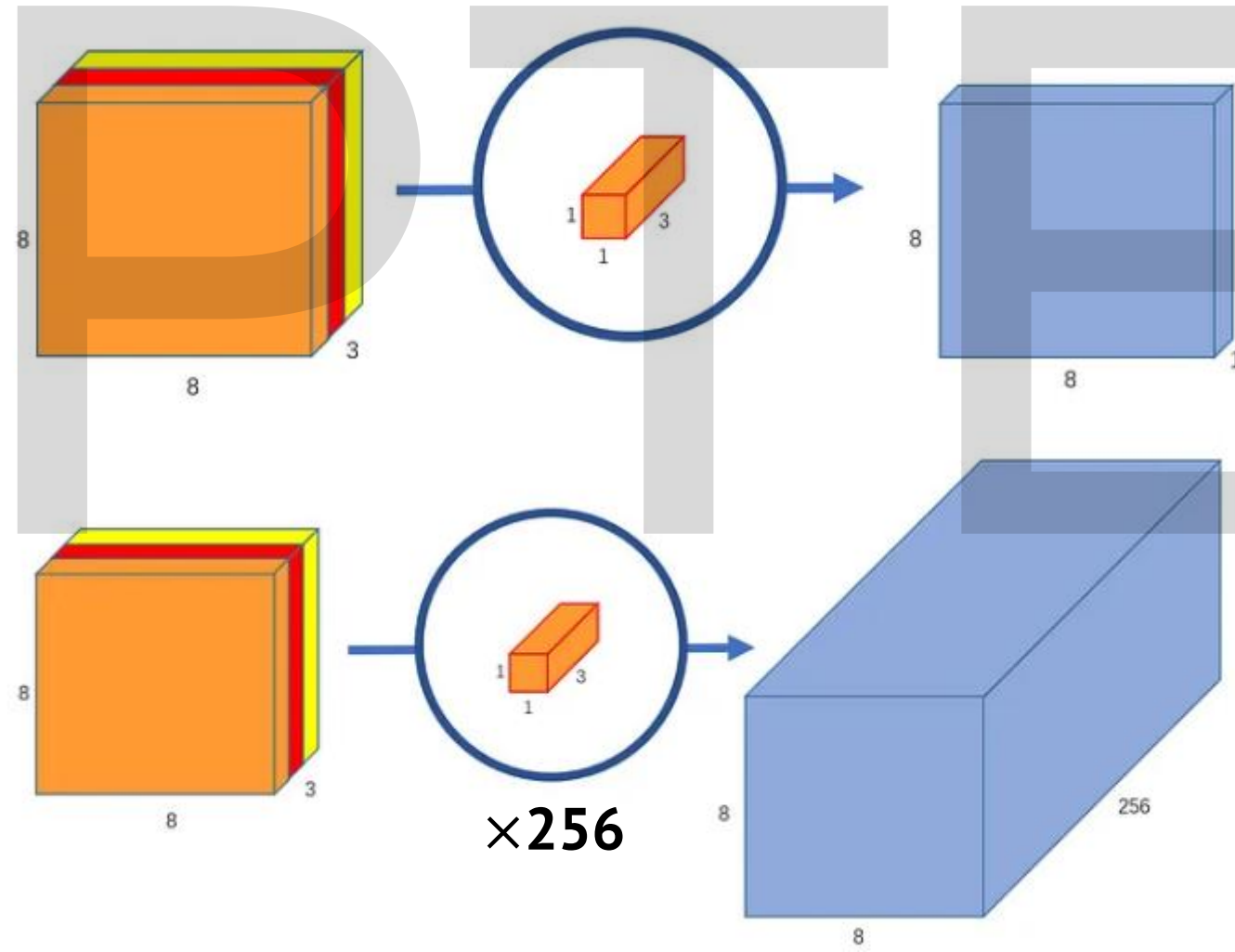
Depth-wise Separable Convolution

- Step-1: Depth-wise Convolution



Depth-wise Separable Convolution

- Step-2: Point-wise Convolution



Agenda

- Recap
- **Dataflow for DNN Execution**

NPTEL

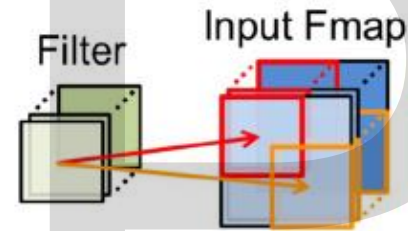
Memory Access Bottleneck for DNNs

- **Worst case**
 - All memory Read/Write are DRAM access
- **AlexNet has 724M MACs**
 - **2896M** DRAM accesses are required
- **Target: Data reuse**

Types of Data Reuse in DNN

Convolutional Reuse

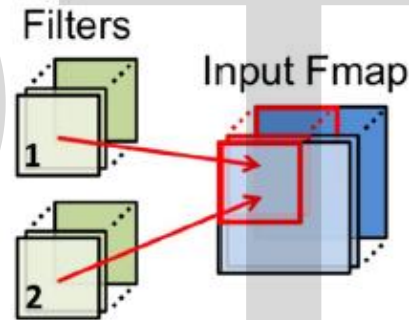
CONV layers only
(sliding window)



Reuse: **Activations**
Filter weights

Fmap Reuse

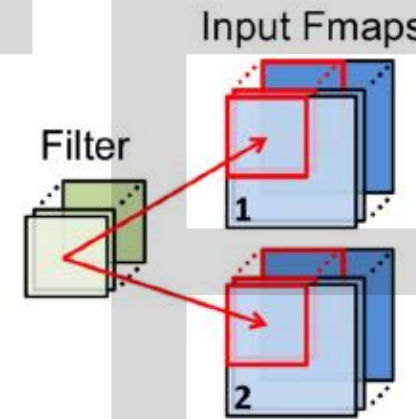
CONV and FC layers



Reuse: **Activations**

Filter Reuse

CONV and FC layers
(batch size > 1)



Reuse: **Filter weights**

- If all types of data reuse is utilized, DRAM access in AlexNet can be reduced from **2896M** to **61M** (best and hypothetical case)

An Example of Systolic Data-flow

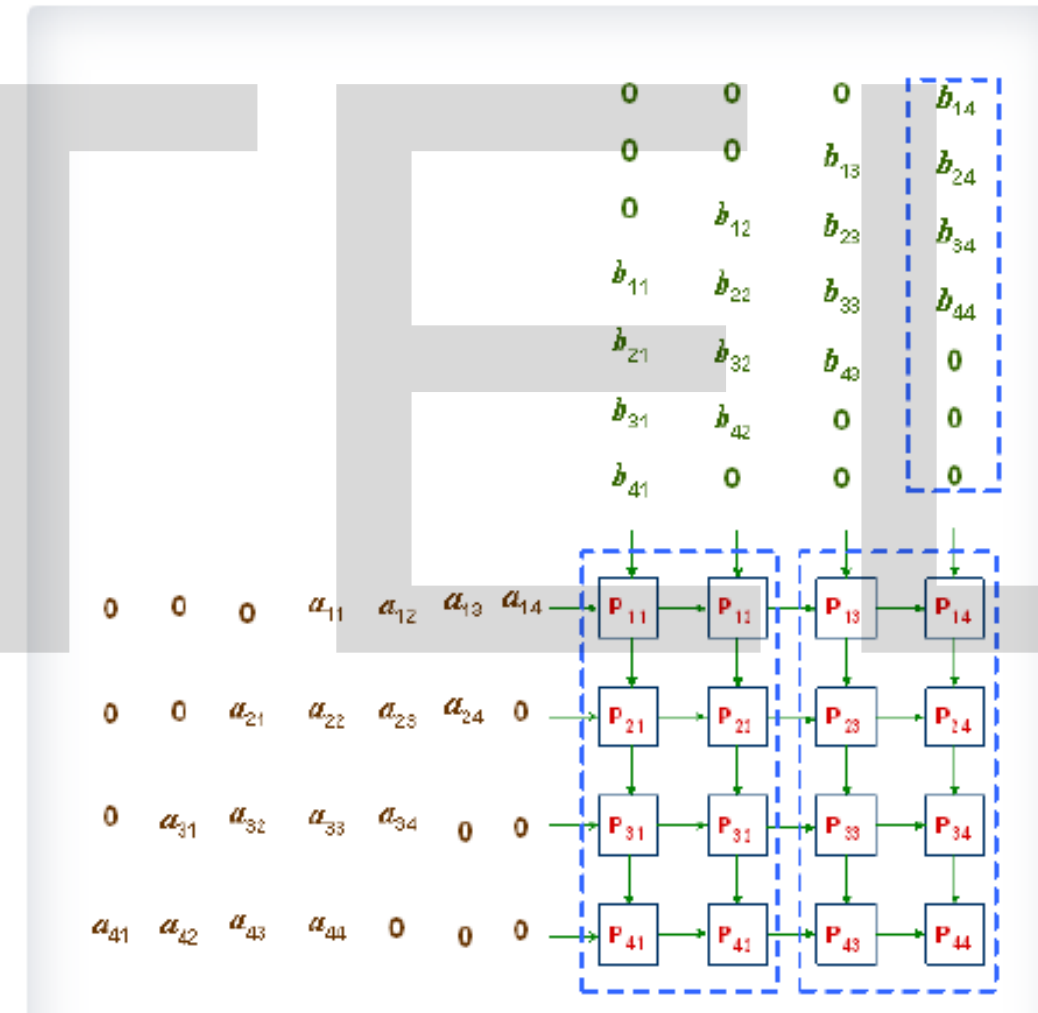
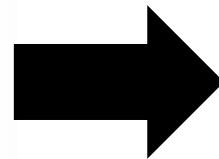
$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix} \times \begin{bmatrix} b_{11} & b_{12} & b_{13} & b_{14} \\ b_{21} & b_{22} & b_{23} & b_{24} \\ b_{31} & b_{32} & b_{33} & b_{34} \\ b_{41} & b_{42} & b_{43} & b_{44} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} & c_{13} & c_{14} \\ c_{21} & c_{22} & c_{23} & c_{24} \\ c_{31} & c_{32} & c_{33} & c_{34} \\ c_{41} & c_{42} & c_{43} & c_{44} \end{bmatrix}$$

$$\begin{aligned} c_{11} &= a_{11} \bullet b_{11} + a_{12} \bullet b_{21} + a_{13} \bullet b_{31} + a_{14} \bullet b_{41} \\ c_{12} &= a_{11} \bullet b_{12} + a_{12} \bullet b_{22} + a_{13} \bullet b_{32} + a_{14} \bullet b_{42} \\ c_{13} &= a_{11} \bullet b_{13} + a_{12} \bullet b_{23} + a_{13} \bullet b_{33} + a_{14} \bullet b_{43} \\ c_{14} &= a_{11} \bullet b_{14} + a_{12} \bullet b_{24} + a_{13} \bullet b_{34} + a_{14} \bullet b_{44} \\ c_{21} &= a_{21} \bullet b_{11} + a_{22} \bullet b_{21} + a_{23} \bullet b_{31} + a_{24} \bullet b_{41} \\ c_{22} &= a_{21} \bullet b_{12} + a_{22} \bullet b_{22} + a_{23} \bullet b_{32} + a_{24} \bullet b_{42} \\ c_{23} &= a_{21} \bullet b_{13} + a_{22} \bullet b_{23} + a_{23} \bullet b_{33} + a_{24} \bullet b_{43} \\ c_{24} &= a_{21} \bullet b_{14} + a_{22} \bullet b_{24} + a_{23} \bullet b_{34} + a_{24} \bullet b_{44} \\ c_{31} &= a_{31} \bullet b_{11} + a_{32} \bullet b_{21} + a_{33} \bullet b_{31} + a_{34} \bullet b_{41} \\ c_{32} &= a_{31} \bullet b_{12} + a_{32} \bullet b_{22} + a_{33} \bullet b_{32} + a_{34} \bullet b_{42} \\ c_{33} &= a_{31} \bullet b_{13} + a_{32} \bullet b_{23} + a_{33} \bullet b_{33} + a_{34} \bullet b_{43} \\ c_{34} &= a_{31} \bullet b_{14} + a_{32} \bullet b_{24} + a_{33} \bullet b_{34} + a_{34} \bullet b_{44} \\ c_{41} &= a_{41} \bullet b_{11} + a_{42} \bullet b_{21} + a_{43} \bullet b_{31} + a_{44} \bullet b_{41} \\ c_{42} &= a_{41} \bullet b_{12} + a_{42} \bullet b_{22} + a_{43} \bullet b_{32} + a_{44} \bullet b_{42} \\ c_{43} &= a_{41} \bullet b_{13} + a_{42} \bullet b_{23} + a_{43} \bullet b_{33} + a_{44} \bullet b_{43} \\ c_{44} &= a_{41} \bullet b_{14} + a_{42} \bullet b_{24} + a_{43} \bullet b_{34} + a_{44} \bullet b_{44} \end{aligned}$$

An Example of Systolic Data-flow

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix} \times \begin{bmatrix} b_{11} & b_{12} & b_{13} & b_{14} \\ b_{21} & b_{22} & b_{23} & b_{24} \\ b_{31} & b_{32} & b_{33} & b_{34} \\ b_{41} & b_{42} & b_{43} & b_{44} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} & c_{13} & c_{14} \\ c_{21} & c_{22} & c_{23} & c_{24} \\ c_{31} & c_{32} & c_{33} & c_{34} \\ c_{41} & c_{42} & c_{43} & c_{44} \end{bmatrix}$$

$$\begin{aligned} c_{11} &= a_{11} \cdot b_{11} + a_{12} \cdot b_{21} + a_{13} \cdot b_{31} + a_{14} \cdot b_{41} \\ c_{12} &= a_{11} \cdot b_{12} + a_{12} \cdot b_{22} + a_{13} \cdot b_{32} + a_{14} \cdot b_{42} \\ c_{13} &= a_{11} \cdot b_{13} + a_{12} \cdot b_{23} + a_{13} \cdot b_{33} + a_{14} \cdot b_{43} \\ c_{14} &= a_{11} \cdot b_{14} + a_{12} \cdot b_{24} + a_{13} \cdot b_{34} + a_{14} \cdot b_{44} \\ c_{21} &= a_{21} \cdot b_{11} + a_{22} \cdot b_{21} + a_{23} \cdot b_{31} + a_{24} \cdot b_{41} \\ c_{22} &= a_{21} \cdot b_{12} + a_{22} \cdot b_{22} + a_{23} \cdot b_{32} + a_{24} \cdot b_{42} \\ c_{23} &= a_{21} \cdot b_{13} + a_{22} \cdot b_{23} + a_{23} \cdot b_{33} + a_{24} \cdot b_{43} \\ c_{24} &= a_{21} \cdot b_{14} + a_{22} \cdot b_{24} + a_{23} \cdot b_{34} + a_{24} \cdot b_{44} \\ c_{31} &= a_{31} \cdot b_{11} + a_{32} \cdot b_{21} + a_{33} \cdot b_{31} + a_{34} \cdot b_{41} \\ c_{32} &= a_{31} \cdot b_{12} + a_{32} \cdot b_{22} + a_{33} \cdot b_{32} + a_{34} \cdot b_{42} \\ c_{33} &= a_{31} \cdot b_{13} + a_{32} \cdot b_{23} + a_{33} \cdot b_{33} + a_{34} \cdot b_{43} \\ c_{34} &= a_{31} \cdot b_{14} + a_{32} \cdot b_{24} + a_{33} \cdot b_{34} + a_{34} \cdot b_{44} \\ c_{41} &= a_{41} \cdot b_{11} + a_{42} \cdot b_{21} + a_{43} \cdot b_{31} + a_{44} \cdot b_{41} \\ c_{42} &= a_{41} \cdot b_{12} + a_{42} \cdot b_{22} + a_{43} \cdot b_{32} + a_{44} \cdot b_{42} \\ c_{43} &= a_{41} \cdot b_{13} + a_{42} \cdot b_{23} + a_{43} \cdot b_{33} + a_{44} \cdot b_{43} \\ c_{44} &= a_{41} \cdot b_{14} + a_{42} \cdot b_{24} + a_{43} \cdot b_{34} + a_{44} \cdot b_{44} \end{aligned}$$

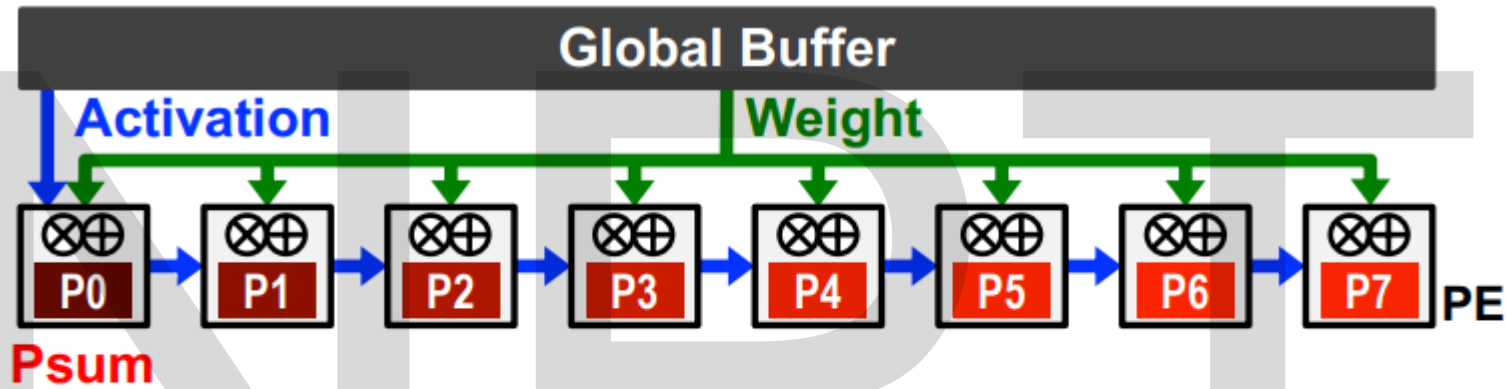


Different Types of Data Flow

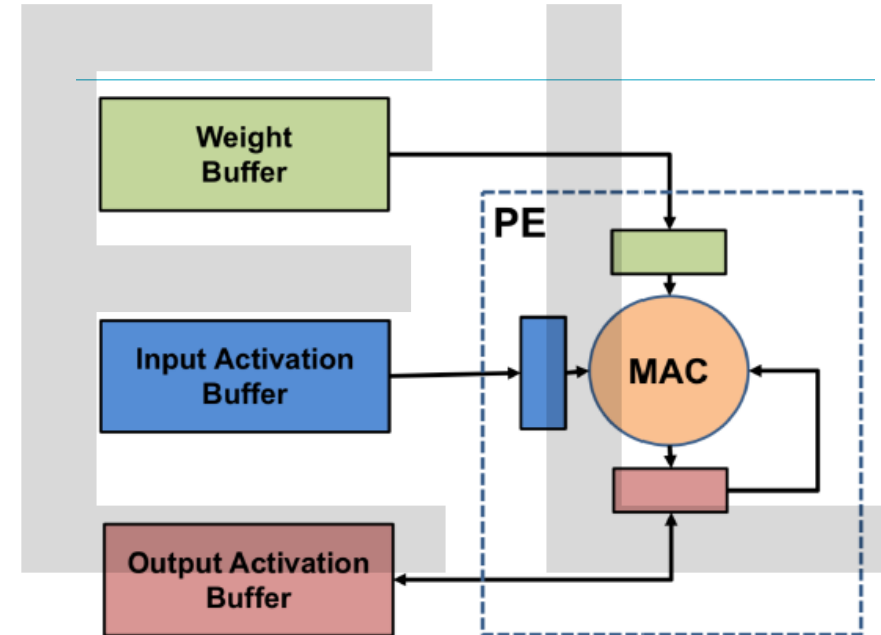
- **Output Stationary (OS)**
- **Weight Stationary (WS)**
- **Input Stationary (IS)**

NPTEL

Output Stationary



- Minimize **partial sum** R/W energy consumption
 - maximize local accumulation
- Broadcast/Multicast **filter weights** and reuse **activations** spatially across the PE array



1-D Convolution – Output Stationary



```
int I[H]; // Input activations
int W[R]; // Filter weights
int O[E]; // Output activations
```

```
for (e = 0; e < E; e++)
  for (r = 0; r < R; r++)
    O[e] += I[e+r] * W[r];
```

No constraints
on loop
permutations!

1-D Convolution – Output Stationary



```
int I[H]; // Input activations
int W[R]; // Filter weights
int O[E]; // Output activations
```

```
for (e = 0; e < E; e++)
  for (r = 0; r < R; r++)
    O[e] += I[e+r] * W[r];
```

No constraints
on loop
permutations!

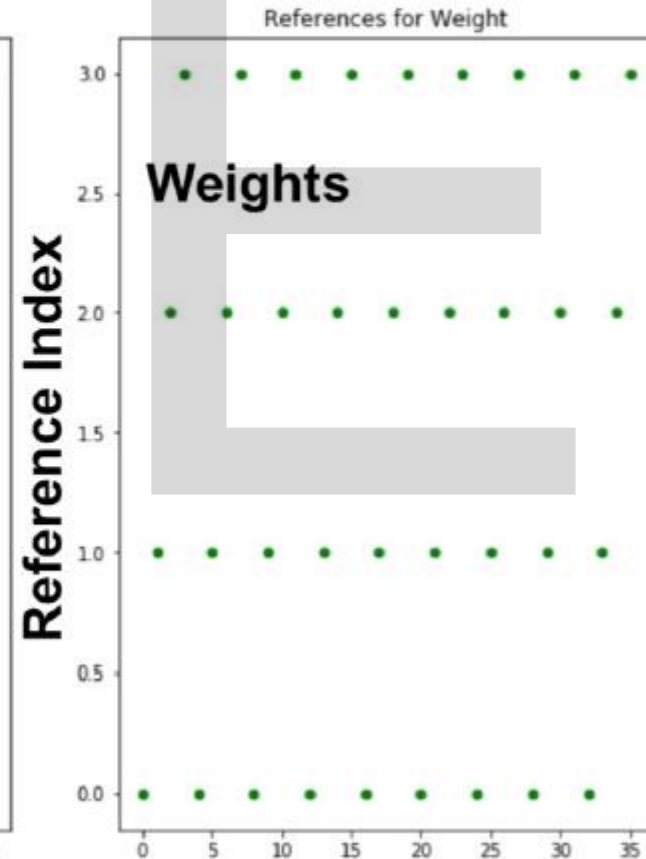
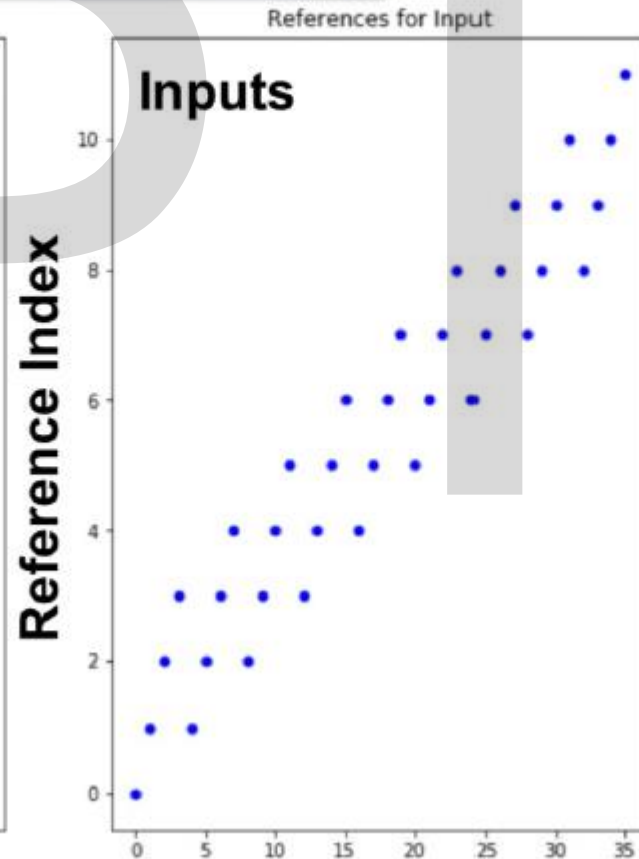
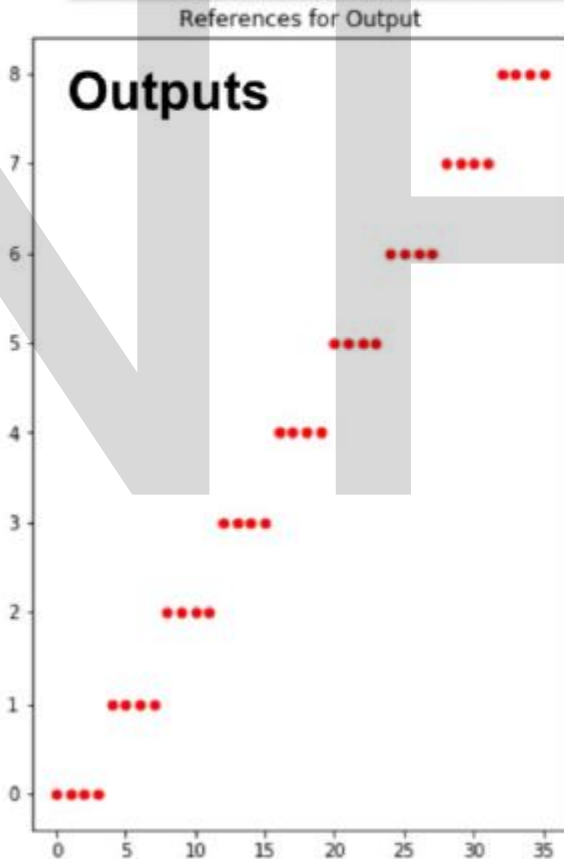
- **For the above loop, we essentially duplicate the weights on all PEs (outer loop)**
 - After duplication, each PE can execute the inner loop
 - The input activations (I) in one window (red box) go over all PEs, then the execution moves on to the next window
 - When all windows are completed, each PE gets the outputs locally

Output Stationary – Reference Pattern

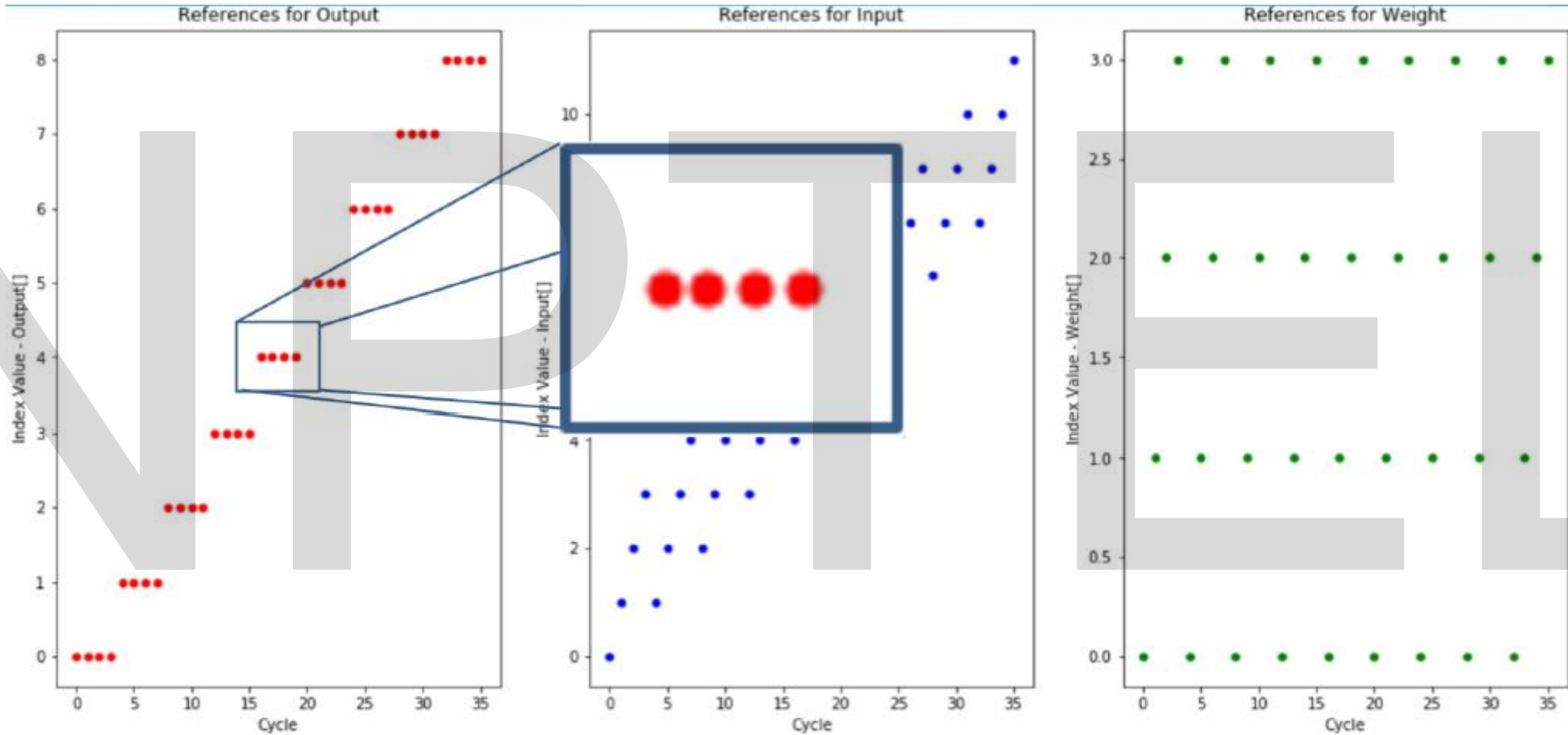
```
for (e = 0; e < E; e++)  
  for (r = 0; r < R; r++)  
    O[e] += I[e+r] * W[r];
```

Layer Shape:

- H = 12
- R = 4
- E = 9



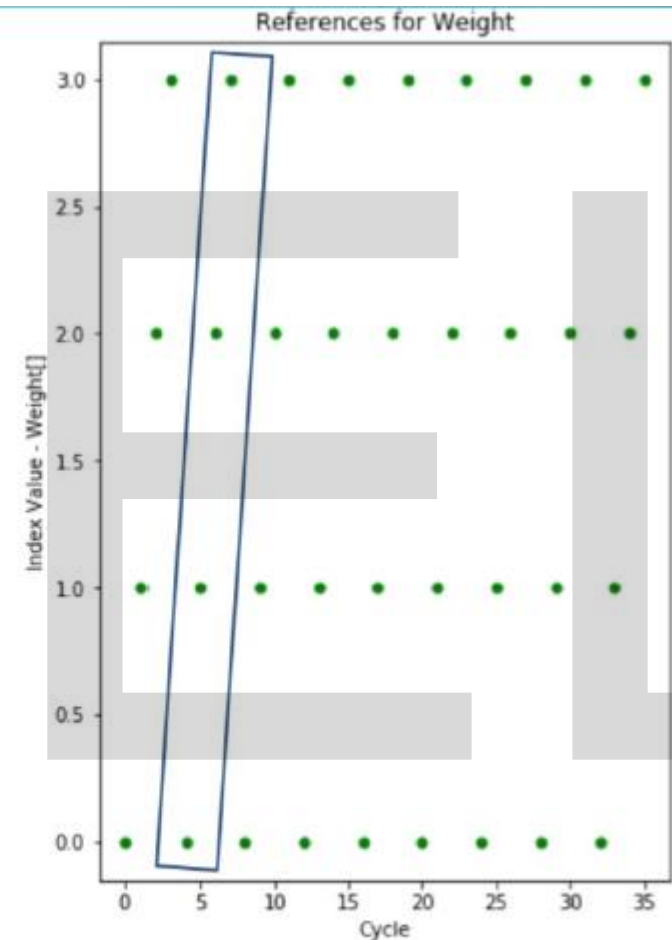
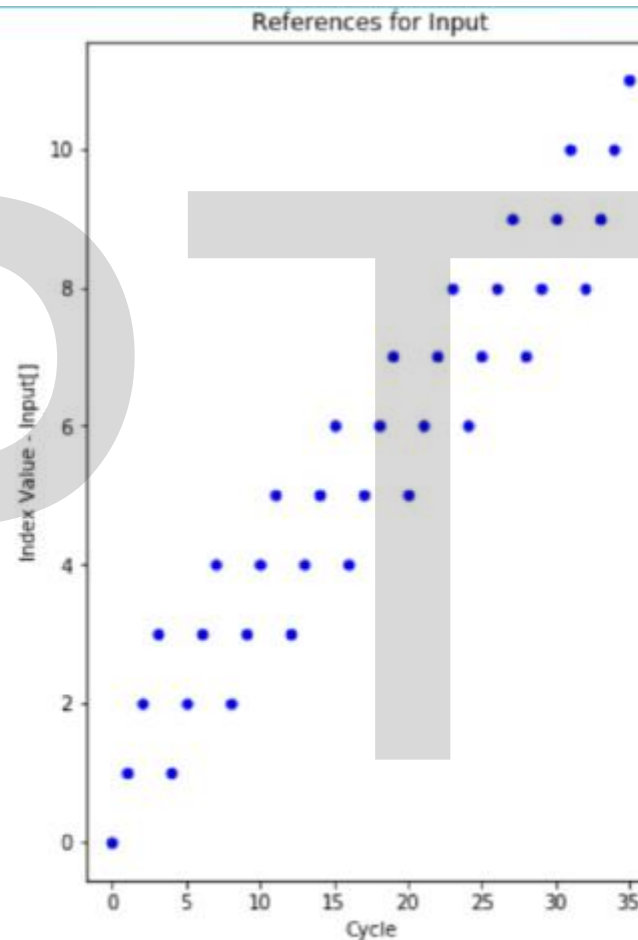
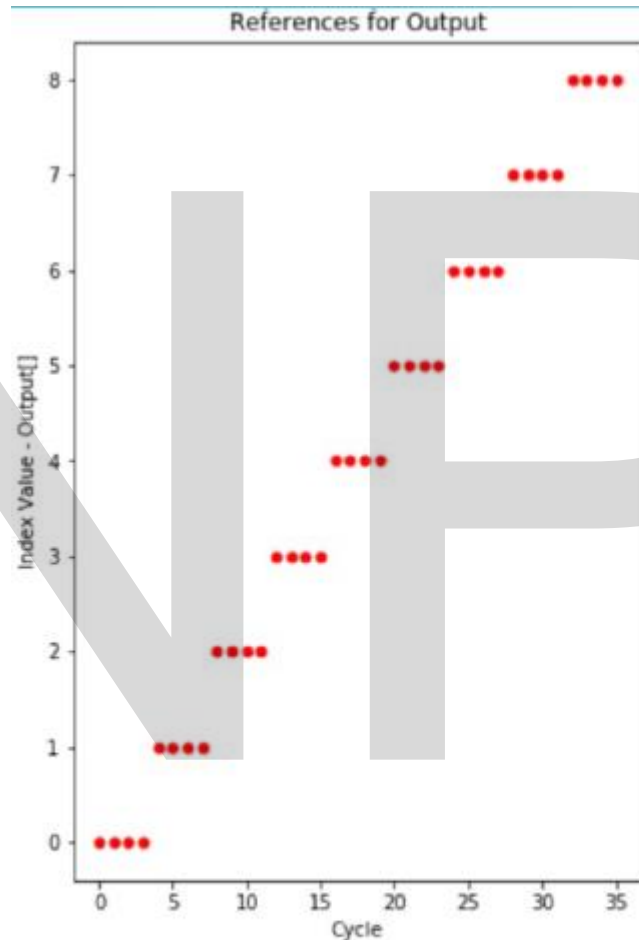
Output Stationary – Reference Pattern



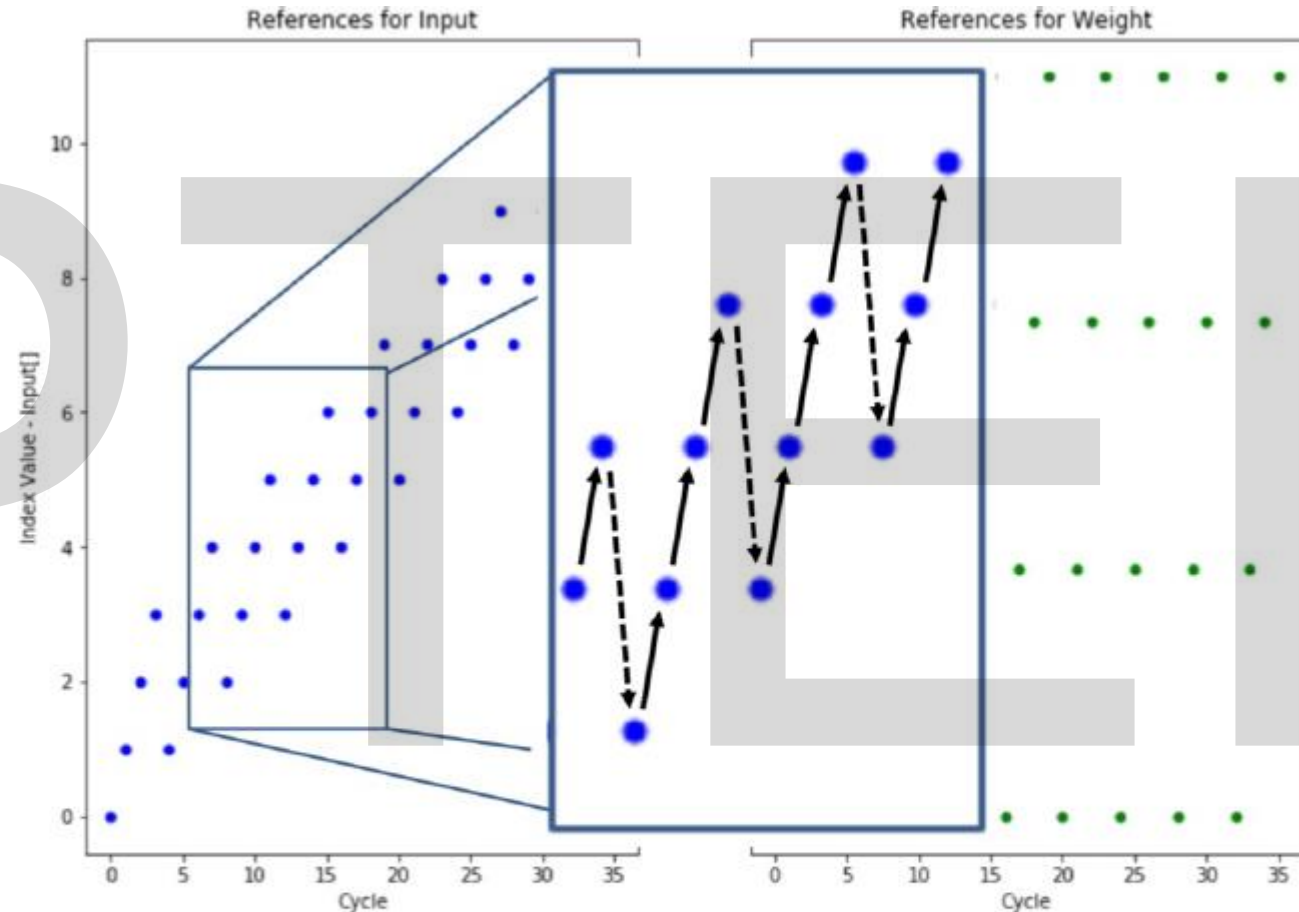
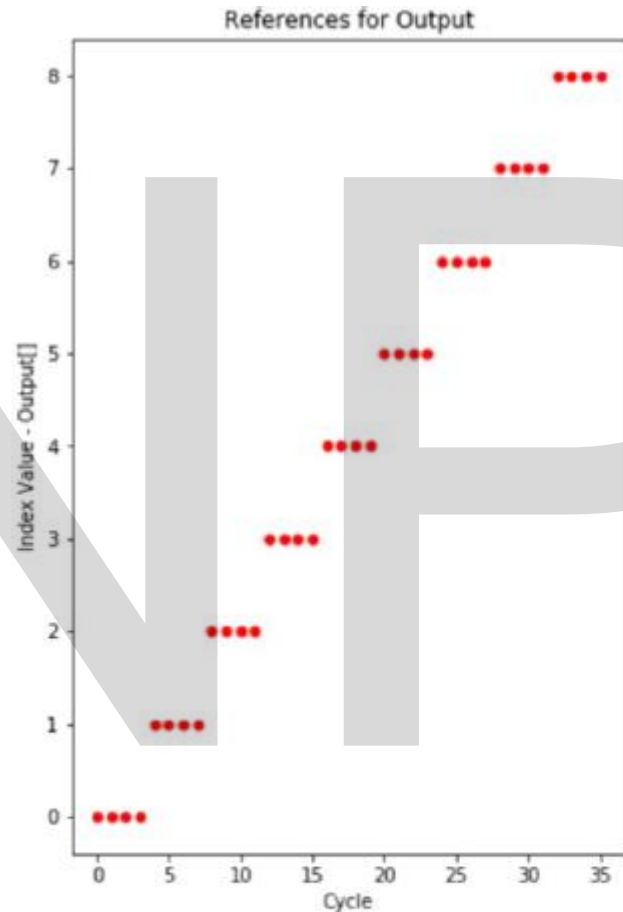
Observations:

- Single **output** is reused many times (R)

Output Stationary – Reference Pattern



Output Stationary – Reference Pattern



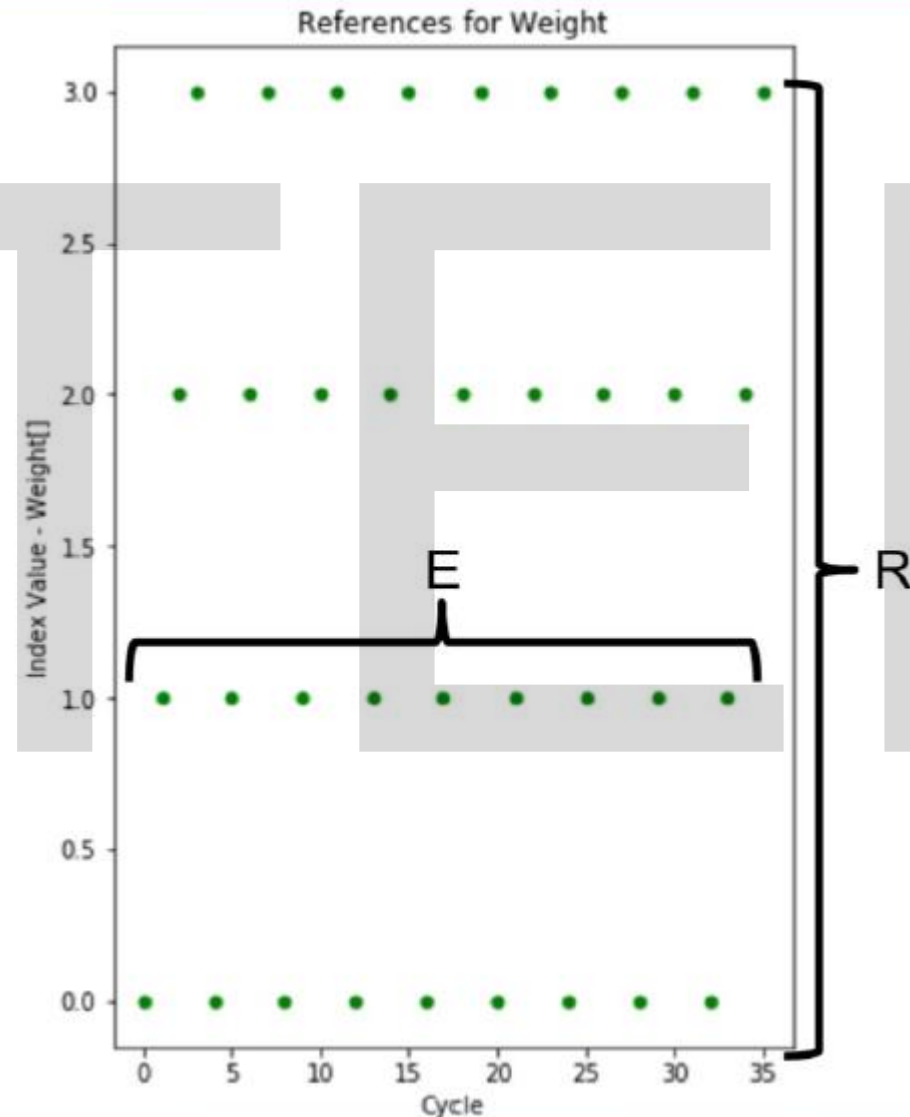
Observations:

- Single **output** is reused many times (R)
- All **weights** reused repeatedly
- Sliding window of **inputs** (size = R)

Output Stationary – Buffer Data Access (Weights)

```
for (e = 0; e < E; e++)  
  for (r = 0; r < R; r++)  
    O[e] += I[e+r] * W[r];
```

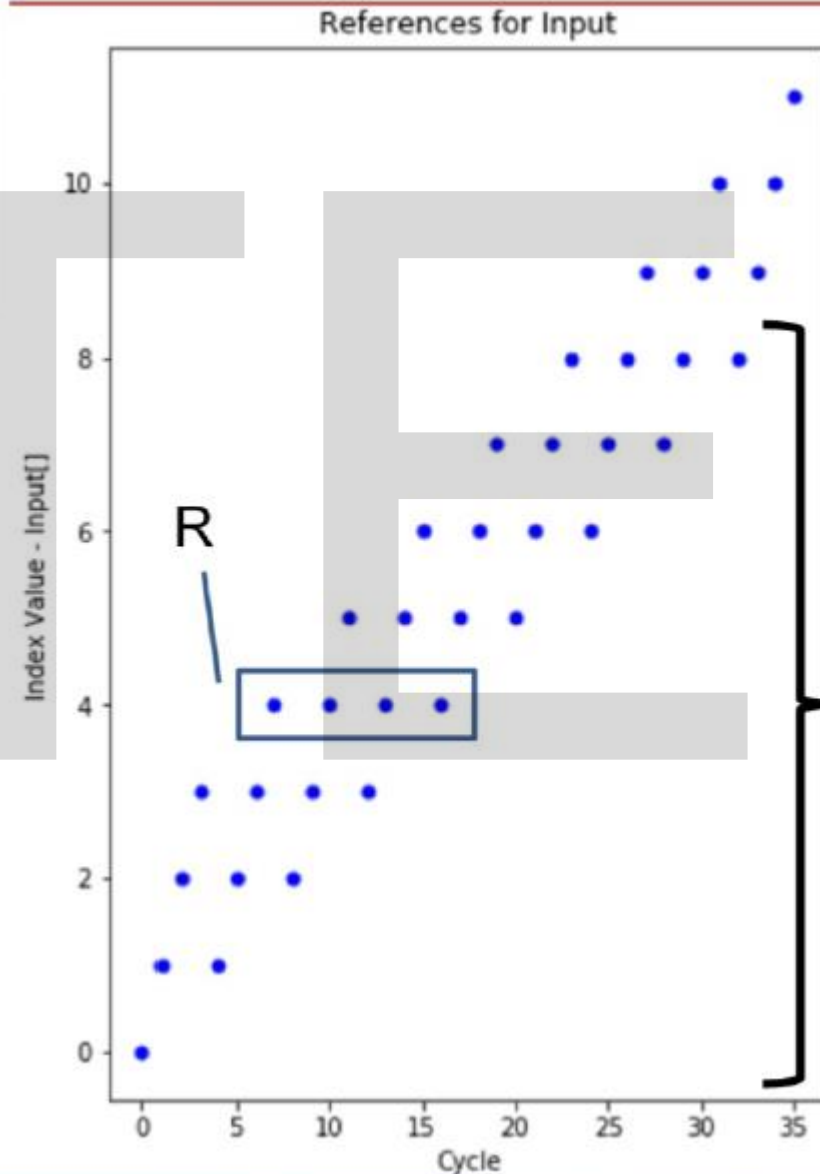
	OS
MACs	$E \cdot R$
Weight Reads	$E \cdot R$
Input Reads	
Output Reads	
Output Writes	



Output Stationary – Buffer Data Access (Inputs)

```
for (e = 0; e < E; e++)  
  for (r = 0; r < R; r++)  
    O[e] += I[e+r] * W[r];
```

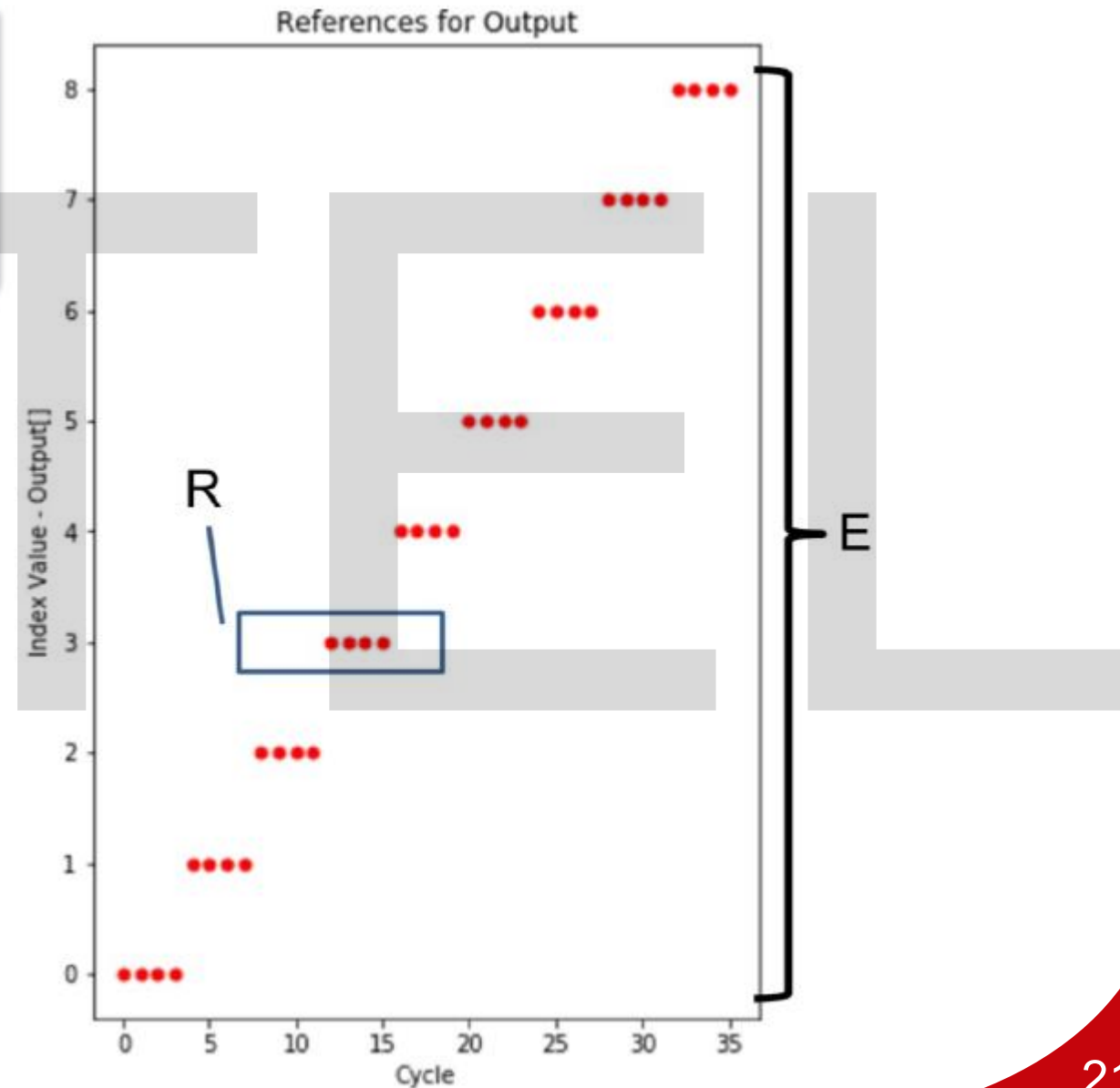
	OS
MACs	$E \cdot R$
Weight Reads	$E \cdot R$
Input Reads	$E \cdot R$
Output Reads	
Output Writes	



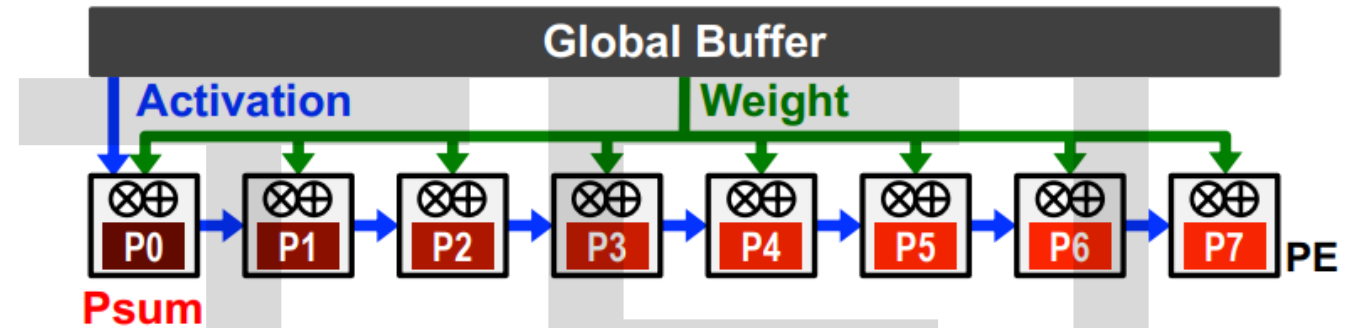
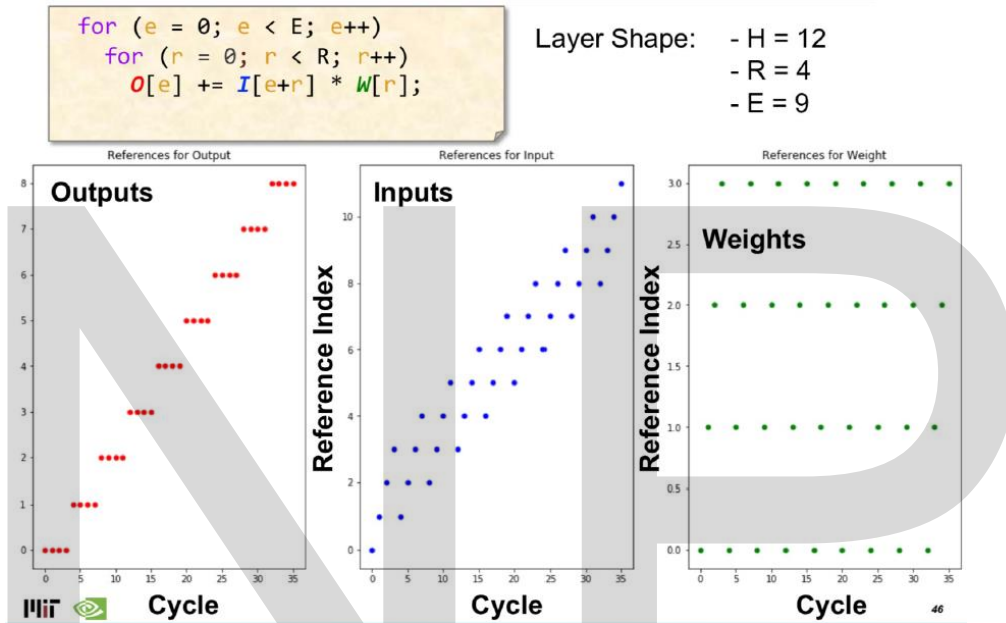
Output Stationary – Buffer Data Access (Outputs)

```
for (e = 0; e < E; e++)  
  for (r = 0; r < R; r++)  
    O[e] += I[e+r] * W[r];
```

	OS
MACs	$E \cdot R$
Weight Reads	$E \cdot R$
Input Reads	$E \cdot R$
Output Reads	0
Output Writes	E



Relation to OS Architecture



- Weights are broadcasted to all PEs, so each one has $W[0]$, $W[1]$, $W[2]$, $W[3]$ when they are needed
- Then, according to the sliding windows in inputs, distribute the required inputs to all PEs
 - Each PE has different four inputs
 - P0: $I[0]$, $I[1]$, $I[2]$, $I[3]$
 - P1: $I[1]$, $I[2]$, $I[3]$, $I[4]$...
- Perform parallel computations to generate outputs
 - P0: $O[0]$, P1: $O[1]$, ...



THANK YOU

Slide Courtesy: Murali Annavaram (University of Southern California), R. Govindarajan (IISc), Arkaprava Basu (IISc), Jayashree Mohan (Microsoft Research)

